

ARES Protocol — Use Cases

The same battery, real commands: JSON, XML, ROS2, and mixed

One protocol, several transports — every command shown in full. The register / heartbeat / claim / release battery runs over **REST/JSON** (`curl`), over **REST/XML** (`curl`), over **ROS2** (`ros2 service call` through `zenoh-bridge-ros2dds`), and **mixed** across all three, against one coordinator and one 7-zone workspace (`dock_a=1 ... dock_c=3`, `zone_4=4 ... zone_7=7`). Zones 1–4 and 6 are exclusive; zones 5 and 7 are shared with capacity 2. Server version `0.1.3`. Every reply is captured from a live run.

Auto-release: a robot loses its zones when it stops heart-beating — no manual reset. Register with a high `<alive>` (e.g. `9999`) to avoid constant heartbeats; the server reclaims a zone only after `2×` the `alive` interval passes with no heartbeat (section F uses a short `alive` to show it).

Leases: `<lease_time>` is in **seconds of wall clock**. A claim carrying one is dropped that many seconds after it is granted, whether or not the robot keeps heart-beating; omitting it (or sending `0`) holds the zone until it is released or the robot goes quiet.

Reason codes (after `0` ok / `1` mismatched key): register `2` already registered, `3` bad id, `5` not permitted (id not provisioned, or keyless registration disabled), `6` fleet full. Heartbeat `2` not registered. Claim `2` conflict, `3` capacity, `4` unknown id, `5` bad request. Release `2` no such lease.

1 JSON (REST) — `curl`

Set `B=http://<host>:8080/ares/v1`. JSON is the default wire format: the `content-type` header below is explicit for clarity, but a body with no `content-type` is read as JSON and a request with no `Accept` is answered in JSON. Replies shown after `# ->`.

1.1 A. Read (no key)

```
curl -s "$B/health"
# -> {"status":"ok","version":"0.1.3","datum":{"lat":52.0,"lon":5.0,"alt":0.0}}
curl -s "$B/zones"          # -> [ ... 8 zones: fixed_yard, dock_a ... dock_c, zone_4 ... zone_7 ]
curl -s "$B/zones/3"
# -> {"id":"...0103","numeric_id":3,"name":"dock_c","kind":"zone",...,
#     "properties":{"traffic.policy":"exclusive",...}}
curl -s "$B/zones/99"      # -> {"message":"unknown zone id Numeric(99)"}
curl -s "$B/fleet/snapshot"
# -> {"robots":[],"requests":[],"leases":[],"inactive_robot_ids":[],"events":[],...}
```

1.2 B. Register

```
curl -s -X POST "$B/robots" -H 'content-type: application/json' \
  -d '{"robot":"91","key":"1234","alive":9999}'
# -> {"decision":1,"reason":0}
curl -s -X POST "$B/robots" -H 'content-type: application/json' \
  -d '{"robot":"91","key":"1234"}'          # again -> already registered
# -> {"decision":0,"reason":2}
curl -s -X POST "$B/robots" -H 'content-type: application/json' \
  -d '{"robot":"92"}'                      # no key -> default
# -> {"decision":1,"reason":0}
curl -s -X POST "$B/robots" -H 'content-type: application/json' \
  -d '{"robot":"abc"}'                    # bad id
# -> {"decision":0,"reason":3}
```

1.3 C. Heartbeat

```
curl -s -X POST "$B/robots/91/heartbeat" -H 'content-type: application/json' \
-d '{"key":"1234","zone":3}' # ok
# -> {"decision":1,"reason":0}
curl -s -X POST "$B/robots/91/heartbeat" -H 'content-type: application/json' \
-d '{"key":"9999","zone":3}' # wrong key
# -> {"decision":0,"reason":1}
curl -s -X POST "$B/robots/999/heartbeat" -H 'content-type: application/json' \
-d '{"zone":3}' # not registered
# -> {"decision":0,"reason":2}
curl -s -X POST "$B/robots/91/heartbeat" -H 'content-type: application/json' \
-d '{"key":"1234","zone":-1}' # location unknown
# -> {"decision":1,"reason":0}
```

1.4 D. Claim

Setup: `curl -s -X POST "$B/robots" -H 'content-type: application/json' -d '{"robot":"94","key":"4444","alive":9999}'`

`id` is an **array** even for a single resource — `[5]`, not `5`.

```
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
-d '{"key":"1234","robot":"91","id":[5]}' # granted
# -> {"decision":1,"reason":0}
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
-d '{"robot":"92","id":[5]}' # 5 held by 91
# -> {"decision":0,"reason":2,"blocked":5}
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
-d '{"robot":"92","id":[6,5]}' # atomic; 5 held
# -> {"decision":0,"reason":2,"blocked":5}
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
-d '{"robot":"92","id":[6]}' # granted
# -> {"decision":1,"reason":0}
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
-d '{"key":"4444","robot":"94","id":[7],"access_mode":1,"lease_time":30}'
# -> {"decision":1,"reason":0} # exclusive + lease
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
-d '{"key":"1234","robot":"91","id":[1],"access_mode":2}' # access_mode 2 = shared
# -> {"decision":1,"reason":0}
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
-d '{"key":"1234","robot":"91","id":[2],"access_mode":3}' # 3+ reserved
# -> {"decision":0,"reason":5}
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
-d '{"key":"1234","robot":"91","id":[99]}' # unknown zone
# -> {"decision":0,"reason":4,"blocked":99}
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
-d '{"key":"0000","robot":"91","id":[2]}' # wrong key
# -> {"decision":0,"reason":1}
```

1.5 E. Release

`id` here is a **single** value, not an array — one resource is released at a time.

```
curl -s -X POST "$B/leases/release/zone" -H 'content-type: application/json' \
-d '{"key":"1234","robot":"91","id":5}' # released
# -> {"decision":1,"reason":0}
curl -s -X POST "$B/leases/release/zone" -H 'content-type: application/json' \
-d '{"key":"1234","robot":"91","id":5}' # nothing held now
# -> {"decision":0,"reason":2}
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
-d '{"robot":"92","id":[5]}' # now free -> granted
# -> {"decision":1,"reason":0}
```

1.6 F. Auto-release on heartbeat timeout

```
curl -s -X POST "$B/robots" -H 'content-type: application/json' \
  -d '{"robot": "95", "key": "5", "alive": 1}' # short alive
# -> {"decision": 1, "reason": 0}
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
  -d '{"key": "5", "robot": "95", "id": [3]}' # 95 takes zone 3
# -> {"decision": 1, "reason": 0}
curl -s -X POST "$B/robots" -H 'content-type: application/json' \
  -d '{"robot": "96", "key": "6"}'
# -> {"decision": 1, "reason": 0}
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
  -d '{"key": "6", "robot": "96", "id": [3]}' # 95 still holds it
# -> {"decision": 0, "reason": 2, "blocked": 3}
sleep 3 # 95 (alive 1) misses 2x its interval -> inactive, zone 3 auto-released
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
  -d '{"key": "6", "robot": "96", "id": [3]}' # now granted
# -> {"decision": 1, "reason": 0}
```

Same protocol, different clothes. Every decision and reason above is identical to the XML chapter that follows — only the encoding differs. Pick a format per client; the coordinator does not care which one a robot speaks.

2 XML (REST) — curl

Set `B=http://<host>:8080/ares/v1`. Send `content-type: application/xml` and, for the read endpoints, ask for `Accept: application/xml`. Replies shown after `# ->`.

2.1 A. Read (no key)

```
curl -s "$B/health"
# -> {"status": "ok", "version": "0.1.3"}
curl -s "$B/zones" # -> [ ... zones: dock_a, dock_b, dock_c, zone_4 ... zone_7 ]
curl -s "$B/zones/3" # -> {"numeric_id": 3, "name": "dock_c", ..., "traffic.policy": "exclusive"}
curl -s "$B/zones/99" # -> {"message": "unknown zone id Numeric(99)"}
curl -s "$B/fleet/snapshot" # -> {"robots": [], "requests": [], "leases": []}
```

2.2 B. Register

```
curl -s -X POST "$B/robots" -H 'content-type: application/xml' \
  -d '<reg><robot>91</robot><key>1234</key><alive>9999</alive></reg>'
# -> <reply><decision>1</decision><reason>0</reason></reply>
curl -s -X POST "$B/robots" -H 'content-type: application/xml' \
  -d '<reg><robot>91</robot><key>1234</key></reg>' # again -> already registered
# -> <reply><decision>0</decision><reason>2</reason></reply>
curl -s -X POST "$B/robots" -H 'content-type: application/xml' \
  -d '<reg><robot>92</robot></reg>' # no key -> default
# -> <reply><decision>1</decision><reason>0</reason></reply>
curl -s -X POST "$B/robots" -H 'content-type: application/xml' \
  -d '<reg><robot>abc</robot></reg>' # bad id
# -> <reply><decision>0</decision><reason>3</reason></reply>
```

2.3 C. Heartbeat

```
curl -s -X POST "$B/robots/91/heartbeat" -H 'content-type: application/xml' \
  -d '<hb><key>1234</key><zone>3</zone></hb>' # ok
# -> <reply><decision>1</decision><reason>0</reason></reply>
curl -s -X POST "$B/robots/91/heartbeat" -H 'content-type: application/xml' \
  -d '<hb><key>9999</key><zone>3</zone></hb>' # wrong key
# -> <reply><decision>0</decision><reason>1</reason></reply>
curl -s -X POST "$B/robots/999/heartbeat" -H 'content-type: application/xml' \
  -d '<hb><zone>3</zone></hb>' # not registered
```

```
# -> <reply><decision>0</decision><reason>2</reason></reply>
curl -s -X POST "$B/robots/91/heartbeat" -H 'content-type: application/xml' \
  -d '<hb><key>1234</key><zone>-1</zone></hb>' # location unknown
# -> <reply><decision>1</decision><reason>0</reason></reply>
```

2.4 D. Claim

Setup: `curl -s -X POST "$B/robots" -H 'content-type: application/xml' -d '<reg><robot>94</robot><key>4444</key><alive>9999</alive></reg>'`

```
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><key>1234</key><robot>91</robot><id>5</id></claim>' # granted
# -> <reply><decision>1</decision><reason>0</reason></reply>
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><robot>92</robot><id>5</id></claim>' # 5 held by 91
# -> <reply><decision>0</decision><reason>2</reason><blocked>5</blocked></reply>
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><robot>92</robot><id>6</id><id>5</id></claim>' # atomic; 5 held
# -> <reply><decision>0</decision><reason>2</reason><blocked>5</blocked></reply>
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><robot>92</robot><id>6</id></claim>' # granted
# -> <reply><decision>1</decision><reason>0</reason></reply>
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><key>4444</key><robot>94</robot><id>7</id>
    <access_mode>1</access_mode><lease_time>30</lease_time></claim>' # exclusive + lease
# -> <reply><decision>1</decision><reason>0</reason></reply>
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><key>1234</key><robot>91</robot><id>1</id>
    <access_mode>2</access_mode></claim>' # access_mode 2 = shared
# -> <reply><decision>1</decision><reason>0</reason></reply>
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><key>1234</key><robot>91</robot><id>2</id>
    <access_mode>3</access_mode></claim>' # 3+ reserved
# -> <reply><decision>0</decision><reason>5</reason></reply>
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><key>1234</key><robot>91</robot><id>99</id></claim>' # unknown zone
# -> <reply><decision>0</decision><reason>4</reason><blocked>99</blocked></reply>
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><key>0000</key><robot>91</robot><id>2</id></claim>' # wrong key
# -> <reply><decision>0</decision><reason>1</reason></reply>
```

2.5 E. Release

```
curl -s -X POST "$B/leases/release/zone" -H 'content-type: application/xml' \
  -d '<rel><key>1234</key><robot>91</robot><id>5</id></rel>' # released
# -> <reply><decision>1</decision><reason>0</reason></reply>
curl -s -X POST "$B/leases/release/zone" -H 'content-type: application/xml' \
  -d '<rel><key>1234</key><robot>91</robot><id>5</id></rel>' # nothing held now
# -> <reply><decision>0</decision><reason>2</reason></reply>
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><robot>92</robot><id>5</id></claim>' # now free -> granted
# -> <reply><decision>1</decision><reason>0</reason></reply>
```

2.6 F. Auto-release on heartbeat timeout

```
curl -s -X POST "$B/robots" -H 'content-type: application/xml' \
  -d '<reg><robot>95</robot><key>5</key><alive>1</alive></reg>' # short alive
# -> <reply><decision>1</decision><reason>0</reason></reply>
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><key>5</key><robot>95</robot><id>3</id></claim>' # 95 takes zone 3
# -> <reply><decision>1</decision><reason>0</reason></reply>
curl -s -X POST "$B/robots" -H 'content-type: application/xml' \
  -d '<reg><robot>96</robot><key>6</key></reg>'
# -> <reply><decision>1</decision><reason>0</reason></reply>
```

```
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><key>6</key><robot>96</robot><id>3</id></claim>' # 95 still holds it
# -> <reply><decision>0</decision><reason>2</reason><blocked>3</blocked></reply>
sleep 3 # 95 (alive 1) misses 2x its interval -> inactive, zone 3 auto-released
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><key>6</key><robot>96</robot><id>3</id></claim>' # now granted
# -> <reply><decision>1</decision><reason>0</reason></reply>
```

`access_mode 2` now means **shared**, not “reserved” — on a free zone it is granted, and only lets robots coexist on a `shared` -policy zone with capacity > 1. On a free exclusive zone it behaves like a normal claim. `access_mode 3` + is still reserved (→ reason `5`).

3 ROS2 — `ros2 service call`

The generic service type is `ares_interfaces/srv/Json`; the reply is `success=True` with `response=<json>` (shown after `# ->`).

3.1 A. Read

```
ros2 service call /ares/v1/health ares_interfaces/srv/Json "{request: '{}'}"
# -> response: {"status":"ok","version":"0.1.3"}
ros2 service call /ares/v1/zones/get ares_interfaces/srv/Json "{request: '{}\"id\":3\"'}"
# -> response: {"numeric_id":3,"name":"dock_c", ... }
ros2 service call /ares/v1/zones/get ares_interfaces/srv/Json "{request: '{}\"id\":99\"'}"
# -> success=False, response: unknown zone id ...
ros2 service call /ares/v1/fleet/snapshot ares_interfaces/srv/Json "{request: '{}'}"
# -> response: {"robots":[],"requests":[],"leases":[]}
```

3.2 B. Register

```
ros2 service call /ares/v1/robots/register ares_interfaces/srv/Json \
  "{request: '{}\"robot\":91\", \"key\":1234\", \"alive\":9999\"'}"
# -> response: {"decision":1,"reason":0}
ros2 service call /ares/v1/robots/register ares_interfaces/srv/Json \
  "{request: '{}\"robot\":91\", \"key\":1234\"'}" # again
# -> response: {"decision":0,"reason":2}
ros2 service call /ares/v1/robots/register ares_interfaces/srv/Json \
  "{request: '{}\"robot\":92\", \"alive\":9999\"'}" # no key -> default
# -> response: {"decision":1,"reason":0}
ros2 service call /ares/v1/robots/register ares_interfaces/srv/Json \
  "{request: '{}\"robot\":abc\"'}" # bad id
# -> response: {"decision":0,"reason":3}
```

3.3 C. Heartbeat

```
ros2 service call /ares/v1/robots/heartbeat ares_interfaces/srv/Json \
  "{request: '{}\"robot\":91\", \"key\":1234\", \"zone\":3\"'}" # ok
# -> response: {"decision":1,"reason":0}
ros2 service call /ares/v1/robots/heartbeat ares_interfaces/srv/Json \
  "{request: '{}\"robot\":91\", \"key\":9999\", \"zone\":3\"'}" # wrong key
# -> response: {"decision":0,"reason":1}
ros2 service call /ares/v1/robots/heartbeat ares_interfaces/srv/Json \
  "{request: '{}\"robot\":999\", \"zone\":3\"'}" # not registered
# -> response: {"decision":0,"reason":2}
ros2 service call /ares/v1/robots/heartbeat ares_interfaces/srv/Json \
  "{request: '{}\"robot\":91\", \"key\":1234\", \"zone\":-1\"'}" # location unknown
# -> response: {"decision":1,"reason":0}
```

3.4 D. Claim (setup: register 94 with key 4444)

```

ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"key\": \"1234\", \"robot\": \"91\", \"id\": [5]}'}" # granted
# -> response: {"decision":1,"reason":0}
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"robot\": \"92\", \"id\": [5]}'}" # 5 held by 91
# -> response: {"decision":0,"reason":2,"blocked":5}
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"robot\": \"92\", \"id\": [6,5]}'}" # atomic; 5 held
# -> response: {"decision":0,"reason":2,"blocked":5}
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"robot\": \"92\", \"id\": [6]}'}" # granted
# -> response: {"decision":1,"reason":0}
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"key\": \"4444\", \"robot\": \"94\", \"id\": [7], \"access_mode\": 1, \"lease_time\": 30}'}"
# -> response: {"decision":1,"reason":0}
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"key\": \"1234\", \"robot\": \"91\", \"id\": [1], \"access_mode\": 2}'}" # shared
# -> response: {"decision":1,"reason":0}
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"key\": \"1234\", \"robot\": \"91\", \"id\": [2], \"access_mode\": 3}'}" # reserved
# -> response: {"decision":0,"reason":5}
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"key\": \"1234\", \"robot\": \"91\", \"id\": [99]}'}" # unknown zone
# -> response: {"decision":0,"reason":4,"blocked":99}
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"key\": \"0000\", \"robot\": \"91\", \"id\": [2]}'}" # wrong key
# -> response: {"decision":0,"reason":1}

```

3.5 E. Release

```

ros2 service call /ares/v1/leases/release/zone ares_interfaces/srv/Json \
  "{request: '{\"key\": \"1234\", \"robot\": \"91\", \"id\": 5}'}" # released
# -> response: {"decision":1,"reason":0}
ros2 service call /ares/v1/leases/release/zone ares_interfaces/srv/Json \
  "{request: '{\"key\": \"1234\", \"robot\": \"91\", \"id\": 5}'}" # nothing held now
# -> response: {"decision":0,"reason":2}
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"robot\": \"92\", \"id\": [5]}'}" # now free -> granted
# -> response: {"decision":1,"reason":0}

```

3.6 F. Auto-release on heartbeat timeout

```

ros2 service call /ares/v1/robots/register ares_interfaces/srv/Json \
  "{request: '{\"robot\": \"95\", \"key\": \"5\", \"alive\": 5}'}" # short alive
# -> response: {"decision":1,"reason":0}
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"key\": \"5\", \"robot\": \"95\", \"id\": [3]}'}" # 95 takes zone 3
# -> response: {"decision":1,"reason":0}
ros2 service call /ares/v1/robots/register ares_interfaces/srv/Json \
  "{request: '{\"robot\": \"96\", \"key\": \"6\", \"alive\": 9999}'}"
# -> response: {"decision":1,"reason":0}
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"key\": \"6\", \"robot\": \"96\", \"id\": [3]}'}" # 95 still holds it
# -> response: {"decision":0,"reason":2,"blocked":3}
sleep 15 # 95 (alive 5) misses 2x its interval -> inactive, zone 3 auto-released
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"key\": \"6\", \"robot\": \"96\", \"id\": [3]}'}" # now granted
# -> response: {"decision":1,"reason":0}

```

4 Mixed (REST/JSON + REST/XML + ROS2, one coordinator)

Three robots on **different transports** against the **same** server: robot **91** speaks REST/XML (`curl`), robot **92** speaks ROS2 (`ros2 service call`), and robot **93** speaks REST/JSON (`curl`). We register one on each, then run the experiments — a claim on any of them blocks the others, in every direction, and a release on one frees the zone for the rest.

```
# --- register: 91 over XML, 92 over ROS2 ---
curl -s -X POST "$B/robots" -H 'content-type: application/xml' \
  -d '<reg><robot>91</robot><key>1234</key><alive>9999</alive></reg>'
# -> <reply><decision>1</decision><reason>0</reason></reply>
ros2 service call /ares/v1/robots/register ares_interfaces/srv/Json \
  "{request: {'\robot\':"92\'",\'key\':"2222\'",\'alive\':9999}'}"
# -> response: {"decision":1,"reason":0}
curl -s -X POST "$B/robots" -H 'content-type: application/json' \
  -d '{"robot":93,"key":3333,"alive":9999}'
# -> {"decision":1,"reason":0}

# --- experiment 1: XML robot 91 takes zone 4, ROS2 robot 92 is blocked ---
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><key>1234</key><robot>91</robot><id>4</id></claim>'
# -> <reply><decision>1</decision><reason>0</reason></reply> (91 holds zone 4)
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: {'\key\':"2222\'",\'robot\':"92\'",\'id\':[4]}'}"
# -> response: {"decision":0,"reason":2,"blocked":4} (blocked by REST 91)

# --- experiment 2: ROS2 robot 92 takes zone 6, XML robot 91 is blocked ---
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: {'\key\':"2222\'",\'robot\':"92\'",\'id\':[6]}'}"
# -> response: {"decision":1,"reason":0} (92 holds zone 6)
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><key>1234</key><robot>91</robot><id>6</id></claim>'
# -> <reply><decision>0</decision><reason>2</reason><blocked>6</blocked></reply> (blocked by ROS2 92)

# --- experiment 3: XML robot 91 releases zone 4, ROS2 robot 92 can now take it ---
curl -s -X POST "$B/leases/release/zone" -H 'content-type: application/xml' \
  -d '<rel><key>1234</key><robot>91</robot><id>4</id></rel>'
# -> <reply><decision>1</decision><reason>0</reason></reply>
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: {'\key\':"2222\'",\'robot\':"92\'",\'id\':[4]}'}"
# -> response: {"decision":1,"reason":0} (now free -> granted)

# --- experiment 4: JSON robot 93 takes zone 2, both others are blocked ---
curl -s -X POST "$B/claims/zone" -H 'content-type: application/json' \
  -d '{"key":3333,"robot":93,"id":2}'
# -> {"decision":1,"reason":0} (93 holds zone 2)
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><key>1234</key><robot>91</robot><id>2</id></claim>'
# -> <reply><decision>0</decision><reason>2</reason><blocked>2</blocked></reply> (blocked by JSON 93)
ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: {'\key\':"2222\'",\'robot\':"92\'",\'id\':[2]}'}"
# -> response: {"decision":0,"reason":2,"blocked":2} (blocked by JSON 93)

# --- experiment 5: JSON robot 93 releases zone 2, XML robot 91 takes it ---
curl -s -X POST "$B/leases/release/zone" -H 'content-type: application/json' \
  -d '{"key":3333,"robot":93,"id":2}'
# -> {"decision":1,"reason":0}
curl -s -X POST "$B/claims/zone" -H 'content-type: application/xml' \
  -d '<claim><key>1234</key><robot>91</robot><id>2</id></claim>'
# -> <reply><decision>1</decision><reason>0</reason></reply> (now free -> granted)
```

Captured live: REST/JSON and REST/XML via `curl`, ROS2 via `ros2 service call` through `zenoh-bridge-ros2dds` on CycloneDDS, all against one coordinator on the same seven-zone workspace.